

枚举与 DFS

qwaszx

2026 年 1 月 16 日

本节课目标

- 学会使用 DFS（深度优先搜索）来实现一般的枚举
- 初步理解图和树的概念，并用于对问题进行建模

问题

从 1 到 n 选两个不同的数 (i, j 和 j, i 算同一种)

问题

从 1 到 n 选两个不同的数 (i, j 和 j, i 算同一种)

```
1 for (int i = 1; i <= n; i++) {  
2     for (int j = i + 1; j <= n; j++) {  
3         // (i, j)  
4     }  
5 }
```

枚举组合：更多的数

问题

从 1 到 n 选**五**个不同的数（不同顺序算同一种）

枚举组合：更多的数

问题

从 1 到 n 选**五**个不同的数（不同顺序算同一种）

```
1 for (int i1 = 1; i1 <= n; i1++) {  
2     for (int i2 = i1 + 1; i2 <= n; i2++) {  
3         for (int i3 = i2 + 1; i3 <= n; i3++) {  
4             for (int i4 = i3 + 1; i4 <= n; i4++) {  
5                 for (int i5 = i4 + 1; i5 <= n; i5++) {  
6                     // (i1, i2, i3, i4, i5)  
7                 }  
8             }  
9         }  
10    }  
11 }
```

枚举组合：更多更多的数

问题

从 1 到 n 选 k 个不同的数（不同顺序算同一种）

枚举组合：更多更多的数

问题

从 1 到 n 选 k 个不同的数（不同顺序算同一种）

怎么办？

在写完前 i 层循环后，我们处在这样的状态：我们已经枚举了 i 个数，还需要枚举剩下的数。

我们接下来枚举第 $i + 1$ 个数是什么，然后进入到下面的子问题：我们已经枚举了 $i + 1$ 个数，还需要枚举剩下的数。

抽象出一个函数 $\text{dfs}(t)$ 表示：假设之前已经枚举了 t 个数，这个函数需要枚举完剩下的数。

递归

抽象出一个函数 `dfs(t)` 表示：假设之前已经枚举了 t 个数，这个函数需要枚举完剩下的数。

```
1 void dfs(int t) {  
2     if(t==k) { //枚举完了所有的数  
3         //(a[1], ..., a[k])  
4         return;  
5     }  
6     for(int i=a[t]+1; i<=n; i++) { //枚举第 t+1 个数  
7         a[t+1]=i;  
8         dfs(t+1); //递归枚举剩下的数  
9     }  
10 }
```

在外面调用 (`dfs(0)`)

TODO: 加张图

另一个例子

问题

从 1 到 n 选 k 个不同的数（不同顺序算**不同种**）

另一个例子

问题

从 1 到 n 选 k 个不同的数（不同顺序算**不同种**）

可以先考虑 $k = 5$ 时怎么用多重循环写。

```
1 void dfs(int t) {  
2     if(t==k) {  
3         if(a[1],...,a[k] 互不相同) {  
4             //(a[1],...,a[k])  
5         }  
6         return;  
7     }  
8     for(int i=1;i<=n;i++) {  
9         a[t+1]=i;  
10        dfs(t+1);  
11    }  
12 }
```

$O(n^k \cdot n) \sim O(n^k \cdot n^2)$, 取决于如何判断 $a[1], \dots, a[k]$ 互不相同

写法二

```
1 void dfs(int t)
2 {
3     if(t==k)
4     {
5         //(a[1],...,a[k])
6         return;
7     }
8     for(int i=1;i<=n;i++)
9         if(i 和 a[1],...,a[t] 都不同)
10        {
11            a[t+1]=i;
12            dfs(t+1);
13        }
14 }
```

复杂度?

写法二：复杂度

- $T(t, n) = (n - t)T(t + 1, n) + \sum_{i=1}^n \text{time}(\text{判断 } i \text{ 和 } a[1], \dots, a[t] \text{ 都不同})$
- 后面的求和可以这样算：对于和 $a[1], \dots, a[t]$ 都不同的 i ，每个 i 需要花费 t 的代价；对于 $i \in \{a[1], \dots, a[t]\}$ ，一共需要花费 $1 + 2 + \dots + t = t(t + 1)/2$ 的代价。所以总共是 $t(n + (1 - t)/2) = \Theta(nt)$ 。
- 倒着推过来： $T(k, n) = 1$, $T(k - 1, n) = (n - k + 1) + \Theta(nk) = \Theta(nk)$,
 $T(k - 2, n) = (n - k + 2)\Theta(nk) + \Theta(n(k - 2)) = \Theta((n - k + 2)nk)$, ...,
 $T(0, n) = \Theta(n(n - 1) \cdots (n - k + 2)nk)$
- $k \ll n$ 时是 $\Theta(n^k k)$, $k = n$ 时是 $\Theta(n! \cdot n^2)$
- 这是通过**解递推式**来计算复杂度的方法

写法二：复杂度

太复杂了！有没有其他办法？

- 对于每个 t , 计算 $\text{dfs}(t, n)$ **本身**付出的总代价（也就是不考虑后续递归的代价）
- 到达了 $\text{dfs}(t, n)$ 的 $a[1], \dots, a[t-1]$ 有 $n(n-1)\cdots(n-t+1)$ 种。对于 $t < k$, 每种都要付出 $t(n + (1-t))/2$ 的代价；对于 $t = k$, 要付出 1 的代价。
- 对 $t = 0, \dots, k$ 求和即可得到相同的结果。事实上, $t = k-1$ 这一层的贡献远大于其他 t 。
- 这是通过数**每条语句的执行次数**来计算复杂度的方法

我们可以更聪明地判断 i 和 $a[1], \dots, a[t]$ 都不同。

```
1 void dfs(int t)
2 {
3     if(t==k) {
4         //(a[1], ..., a[k])
5         return;
6     }
7     for(int i=1; i<=n; i++)
8         if(!used[i]) {
9             a[t+1]=i;
10            used[i]=1;
11            dfs(t+1);
12            used[i]=0; // 重要
13            a[t+1]=0;
14        }
15 }
```

写法三：解释

事实上，我们对 $\text{dfs}(t)$ 施加了额外的要求： $\text{dfs}(t)$ 结束时，所有变量的状态都被恢复到和开始前相同。换句话说，在函数的**开始**和**结束**时刻，我们都假设只有 $a[1], \dots, a[t]$ 是存在的，后面的 a 都是**不存在的**（于是对应的 $\text{used}[x]=0$ ）。

在 $\text{dfs}(t)$ 的循环中，我们处于“正在枚举 $a[t+1]$ ”的状态。更具体地说，我们在不断地进行以下步骤：

- 判定能否令 $a[t+1]=i$
- 令 $a[t+1]=i$ ，并处理 $a[t+1]=i$ 造成的影响（这里是 $\text{used}[i]=1$ ）
- 递归进 $\text{dfs}(t+1)$ 枚举 $a[t+2], \dots, a[k]$
- （由于我们的要求，此时的所有变量的状态和 $\text{dfs}(t+1)$ 之前是一样的）
- 清空 $a[t+1]=i$ 造成的影响（这里是 $\text{used}[i]=0$ ）

由于每次循环令 $a[t+1]=i$ 后都做了撤销，所有变量的状态和开始前是一样的

TODO: 加个图

写法三：复杂度

我们把之前判断 i 和 $a[1], \dots, a[t]$ 都不同的复杂度从 $\Theta(t)$ 降到了 $\Theta(1)$ 。所以总复杂度少了一个 k , 变成 $\Theta(n(n-1) \cdots (n-k+2)n)$ 。在 $k \ll n$ 时是 $O(n^k)$, $k = n$ 时是 $\Theta(n! \cdot n)$ 。

启示：提前判断（剪枝）

bonus: 在 $k = n$ 时，有没有办法做到 $\Theta(n!)$?

hint: 如果我们可以做到每层复杂度 $O(n - t)$ 而不是 $\Theta(n)$...

一般的 DFS 框架

```
1 void dfs(int t)
2 {
3     if(枚举完了) {
4         判断/处理/...
5         return;
6     }
7     for(枚举下一步的选项)
8         if(可以选这个) {
9             选这个
10            修改相关的东西
11            dfs(t+1);
12            撤销修改和选择
13        }
14 }
```

一般的撤销怎么做

我们以后学数据结构的时候也会遇到这种问题。

记录：改了什么东西，改之前它是什么

撤销和修改的代价相同

每层的局部变量（包括函数参数 t ）都是独立的。

递归过程中最多占用 层数 $\times$ 每层空间 的空间（数组视为多个变量）

例子：数独

问题

给一个 $n^2 \times n^2$ 数独，找一种填法

数独：在每个还没填的格子中填上一个 1 到 n^2 的整数，要求每行、每列、两条对角线、每个 $n \times n$ 宫格中不能出现重复的数

例子：数独

问题

给一个 $n^2 \times n^2$ 数独，找一种填法

数独：在每个还没填的格子中填上一个 1 到 n^2 的整数，要求每行、每列、两条对角线、每个 $n \times n$ 宫格中不能出现重复的数

一种可能的抽象：dfs(i, j) 表示之前已经填完了前 $i - 1$ 行以及第 i 行的前 j 列，需要填完剩下的空。

```

1 void dfs(int i, int j)
2 {
3     if(i==n*n && j==n*n)
4     {
5         // 输出
6         return;
7     }
8     int curi, curj;
9     计算目前需要枚举的位置 (curi,curj);
10    for(int x=1;x<=n*n;x++)
11        if(a[curi][curj] 可以填 x)
12        {
13            a[curi][curj]=x;
14            处理 a[curi][curj]=x 的影响;
15            dfs(curi,curj);
16            撤销 a[curi][curj]=x 的影响;
17        }
18 }

```

if 里面怎么写

```
1 第 curi 行目前没出现过 x && 第 curj 列目前没出现过 x && n*n 宫格内目前没  
   出现过 x  
2 && ((curi,curj) 不在 \ 对角线上 || \ 对角线上目前没出现过 x)  
3 && ((curi,curj) 不在 / 对角线上 || / 对角线上目前没出现过 x)
```

每个行、列、 $n*n$ 宫格、对角线开一个自己的 used。

怎么知道 $(curi, curj)$ 属于哪个 $n*n$ 宫格/对角线?

方法一：预处理

方法二：一条 / 对角线上的 $i + j$ 都相同，一条
对角线上的 $i - j$ 都相同；一个 $n*n$ 宫格内的 $\lfloor (i - 1)/n \rfloor, \lfloor (j - 1)/n \rfloor$ 都相同

我不知道。但是，最坏情况下一定是 $2^{\Theta(n^4)}$ 。

好消息是，日常生活中碰到的 9*9 数独都能很快跑出来。

事实上，搜索的复杂度通常难以计算，所以现在正式比赛也不会有以搜索为正解的题。

但是由于搜索的玄学速度，作为暴力偶尔有奇效。

例子：迷宫

问题

给一个 $n \times n$ 网格，每个格子是空地或者障碍。问能不能从 $(1, 1)$ 走到 (n, n) （可以上下左右走）

你的做法能处理多大的 n ？

写法一（错误）

```
1 // 假设当前走到了 (i,j)
2 void dfs(int i, int j){
3     if(i==n && j==n) {
4         // 可以
5         return 到 main 函数;
6     }
7     for(枚举一个方向) {
8         计算走这个方向后的坐标 (nxti, nxtj)
9         if(能走) {
10             dfs(nxti, nxtj);
11         }
12     }
13 }
```

发现程序崩溃了。

怎么实现 return 到 main 函数?

开一个全局变量/额外传一个可变参数 flag 记录之前是否已经达到终点。在每次递归调用 dfs 后, 如果 flag=1, 则直接 return。

怎么枚举一个方向并计算走这个方向后的坐标?

方向实际对应一个坐标移动量。例如, 向右移动对应 $(nxti, nextj) = (i, j) + (1, 0)$ 。所以, 开一个常量数组记录每个方向下两个坐标的偏移量即可。

写法二

发现问题在于我们可能走了一个圈。所以加上不走回头路的要求。

```
1 // 假设当前走到了 (i,j)
2 void dfs(int i, int j) {
3     if(i==n && j==n) {
4         // 可以
5         return 到 main 函数;
6     }
7     for(枚举一个方向) {
8         计算走这个方向后的坐标 (nxti, nxtj)
9         if(能走 && !vis[nxti][nxtj]) {
10             vis[nxti][nxtj]=1;
11             dfs(nxti, nxtj);
12             vis[nxti][nxtj]=0;
13         }
14     }
15 }
```

写法三

```
1 // 假设当前走到了 (i,j)
2 void dfs(int i, int j) {
3     if(i==n && j==n) {
4         //可以
5         return 到 main 函数;
6     }
7     for(枚举一个方向) {
8         计算走这个方向后的坐标 (nxti, nxtj)
9         if(能走 && !vis[nxti][nxtj]) {
10             vis[nxti][nxtj]=1;
11             dfs(nxti, nxtj);
12         }
13     }
14 }
```

写法三：解释和复杂度

$\text{dfs}(i, j)$ 现在的意思：假设当前走到了 (i, j) ， $\text{dfs}(i, j)$ 会把能够在不经过进入函数时已走过的位置（即 $\text{vis}[x][y]=1$ ）的前提下走到的位置都走完（并标记 vis ）。此外，如果中间遇到了 (n, n) 就直接退出。

如果 $\text{vis}[\text{nxti}][\text{nxtj}]=1$ ，那么

- 要么 $(\text{nxti}, \text{nxtj})$ 在进入 $\text{dfs}(i, j)$ 前已经走过了
- 要么 $(\text{nxti}, \text{nxtj})$ 在进入 $\text{dfs}(i, j)$ 后走过了
 - 这说明已经存在一条 $(1, 1)$ 到 (i, j) 再到 nxti, nxtj 的路径
 - 从 nxti, nxtj 出发，不经过这条路径上的位置的前提下能到达的所有点都走完了
 - 对于经过 $(i, j) \rightarrow (\text{nxti}, \text{nxtj})$ 这条路径上的位置到达的点，在路径上最后经过的那个点的 dfs 中处理掉了
 - 所以，现在再从 $(\text{nxti}, \text{nxtj})$ 出发不会走到任何新的点

由于每个点只会在 vis 从 0 变到 1 的时候进行了一次 dfs ，总共的时间是 $4n^2$

TODO：加张图

例子：字符串变换

问题

给定字符串 S 和 T ，以及若干条变换规则 $A \mapsto B$ ，表示当 $S = A$ 时可以把 S 变成 B 。问能否通过一系列变换把 S 变成 T 。

例子：字符串变换

问题

给定字符串 S 和 T ，以及若干条变换规则 $A \mapsto B$ ，表示当 $S = A$ 时可以把 S 变成 B 。问能否通过一系列变换把 S 变成 T 。

和上一个问题很相似

```
1 // 假设当前 S=s
2 void dfs(string s)
3 {
4     if(s==T)
5     {
6         //可以
7         return 到 main 函数;
8     }
9     for(枚举 s -> B)
10    {
11        if(!vis[B])
12        {
13            vis[B]=1;
14            dfs(B);
15        }
16    }
17 }
```


回顾：一般的 DFS 框架

```
1 void dfs(int t)
2 {
3     if(枚举完了)
4     {
5         判断/处理/...
6         return;
7     }
8     for(枚举下一步的选项)
9         if(可以选这个)
10        {
11            选这个
12            修改相关的东西
13            dfs(t+1);
14            撤销修改和选择
15        }
16 }
```

在一层 dfs 开始前，我们处于某种**状态**。在这一层 dfs 中，我们会枚举当前状态的**后继状态**。

我们可以抽象出名为**有向图**的模型：

- 有很多点，每个点代表一个状态
- 点之间存在有向的箭头，称为（有向）**边**，表示状态间的后继关系

例如，字符串变换问题就是非常标准的有向图模型。我们需要判断：有向图上是否存在一条从 s 走到 t 的路径？

在迷宫问题中，我们可以把每个空地视为一个点，一个空地可以到达和它相邻的空地。

这时，我们注意到，点的到达是相互的： u 和 v 互为后继状态。

这种模型称为**无向图**：

- 有很多点，每个点代表一个状态
- 点之间存在双向（或者无向）的箭头，称为（无向）**边**，表示状态间存在双向关系

容易发现，无向图可以被有向图表示（使用两条边代表两个方向），反之则不然。

有向图和无向图统称为**图**。

树

在前三个例子中，我们可以把当前的局面（填了一些数）视为一个状态，其后继状态为填上下一个数之后得到的局面。

这时，我们注意到：由于我们是按照某种顺序填数的，所以从根出发恰有一条路径能到达每个状态。

这种模型称为**有根树**：

- 是一个有向图
- 起点，也就是初始状态，称为**根**
- 从根出发到每个点的路径恰有一条

如果把有向图换成无向图，那么得到的模型称为**无根树**。事实上，树还有很多等价的描述，我们以后会学到。

有根树和无根树统称为**树**。

DFS 查找图上路径

```
1 void dfs(int u) {  
2     if(u==终点) {  
3         判断/处理/...  
4         return;  
5     }  
6     for(枚举 u 的后继节点 v)  
7         if(!vis[v]) {  
8             vis[v]=1;  
9             dfs(v);  
10            //无需撤销, 因为走过的无需再走  
11        }  
12 }
```

等到后续学习图论时, 我们会介绍更多的 DFS 在图上的应用。

搜索的不同视角

在前三个例子中，搜索相当于在做**分步枚举**。

在后两个例子中，搜索相当于在**寻找图上路径**。

对于一般的问题，可以从这两种视角出发来考虑怎么写搜索。

这也是抽象思想的一个体现：使用抽象的模型来描述具体的问题，然后在抽象的模型（通常更简单）上思考。

本节课的核心

- DFS：灵活的枚举
- 图和树：常见的模型